

1. The starting logic for the problem without any threading could look like this:

```
#include <iostream>
#include <chrono>
#include <random>
#include <array>

int main(int argc, char const *argv[])
{
    std::array<int, 1000> numbers;

    // Seed the random number generator
    std::random_device rd;
    std::mt19937 gen(rd());
    std::uniform_int_distribution<> distrib(1, 100);

    // Fill the array with random numbers
    for (int& num : numbers) {
        num = distrib(gen);
    }

    // Measure the start time
    auto start = std::chrono::high_resolution_clock::now();

    // // Calculate the sum
    int sum = 0;
    for (int num : numbers) {
        sum += num;
    }

    // Measure the end time
    auto end = std::chrono::high_resolution_clock::now();

    // Calculate the duration
    auto duration = std::chrono::duration_cast<std::chrono::milliseconds>(end - start);

    std::cout << "Sum: " << sum << std::endl;
    std::cout << "Time taken: " << duration.count() << " milliseconds" << std::endl;

    return 0;
}
```

```
}
```

2. Refactoring the calculation directly out to a thread could look like this:

```
#include <iostream>
#include <chrono>
#include <random>
#include <array>
#include <thread>

int main(int argc, char const *argv[])
{
    std::array<int, 1000> numbers;

    // Seed the random number generator
    std::random_device rd;
    std::mt19937 gen(rd());
    std::uniform_int_distribution<> distrib(1, 100);

    // Fill the array with random numbers
    for (int& num : numbers) {
        num = distrib(gen);
    }

    // Measure the start time
    auto start = std::chrono::high_resolution_clock::now();

    // // Calculate the sum
    int sum = 0;
    std::thread worker_thread([&]() {
        for (int num : numbers) {
            sum += num;
        }
    });

    worker_thread.join();
    // Measure the end time
    auto end = std::chrono::high_resolution_clock::now();

    // Calculate the duration
    auto duration = std::chrono::duration_cast<std::chrono::milliseconds>(end - start);
```

```

std::cout << "Sum: " << sum << std::endl;
std::cout << "Time taken: " << duration.count() << " milliseconds" << std::endl;

return 0;
}

```

3. Reworking the solution to use `jthread` to start with could look like this:

```

#include <iostream>
#include <chrono>
#include <random>
#include <array>
#include <thread>

int main(int argc, char const *argv[])
{
    std::array<int, 1000> numbers;

    // Seed the random number generator
    std::random_device rd;
    std::mt19937 gen(rd());
    std::uniform_int_distribution<> distrib(1, 100);

    // Fill the array with random numbers
    for (int& num : numbers) {
        num = distrib(gen);
    }

    // Measure the start time
    auto start = std::chrono::high_resolution_clock::now();

    // // Calculate the sum
    int sum = 0;
    std::jthread worker_thread([&]() {
        for (int num : numbers) {
            sum += num;
        }
    });

    worker_thread.join();

    // Measure the end time

```

```

auto end = std::chrono::high_resolution_clock::now();

// Calculate the duration
auto duration = std::chrono::duration_cast<std::chrono::milliseconds>(end - start);

std::cout << "Sum: " << sum << std::endl;
std::cout << "Time taken: " << duration.count() << " milliseconds" << std::endl;

return 0;
}

```

4. This still calls join. If that call is removed the results will be incorrect.

Sum: 0

Time taken: 0 milliseconds

5. The problem is having the jthread go out of scope and automatically join before printing out the results. Surround the jthread in a block scope to force the destructor.

```

#include <iostream>
#include <chrono>
#include <random>
#include <array>
#include <thread>

int main(int argc, char const *argv[])
{
    std::array<int, 1000> numbers;

    // Seed the random number generator
    std::random_device rd;
    std::mt19937 gen(rd());
    std::uniform_int_distribution<> distrib(1, 100);

    // Fill the array with random numbers
    for (int &num : numbers)
    {
        num = distrib(gen);
    }

    // Measure the start time
    auto start = std::chrono::high_resolution_clock::now();

```

```
// // Calculate the sum
int sum = 0;
{
    std::jthread worker_thread([&]()
                                {
        for (int num : numbers) {
            sum += num;
        } });
}

// Measure the end time
auto end = std::chrono::high_resolution_clock::now();

// Calculate the duration
auto duration = std::chrono::duration_cast<std::chrono::milliseconds>(end - start);

std::cout << "Sum: " << sum << std::endl;
std::cout << "Time taken: " << duration.count() << " milliseconds" << std::endl;

return 0;
}
```